

FindIt Surat React Codex Prompt Guide

A staged build plan for validating the marketplace first, then SEO, then AI visual search

This guide converts the supplied product vision into a safe sequence of small, testable Codex tasks. It intentionally starts with the three essential experiences: sellers publishing products, customers discovering them, and admins approving them. Only after those flows are visible and usable should the team invest in SEO, AI image search, and the wider SaaS feature set.

Key architecture decision

Use React rather than Next.js. For strong SEO, use React Router 7 in framework mode with server-side rendering for public routes, backed by Vite. This is still a React application, but it produces crawlable HTML and per-page metadata. Keep the seller and admin areas authenticated and client-oriented; they do not need search indexing.

What success looks like for the first release

- A seller can create a shop and submit a product with images.
- An admin can approve or reject sellers and products.
- A customer can browse, search, filter, and contact an approved shop by phone or WhatsApp.
- There is no cart, payment, shipping, or online checkout. The conversion is an offline store visit.

Recommended stack and boundaries

Layer	Recommendation	Why it belongs here
Public web	React, TypeScript, React Router 7 SSR, Vite, Tailwind, shadcn/ui	Modern React with crawlable public pages and fast mobile rendering.
API	Node.js, Express, TypeScript	Clear REST boundary shared by public web, seller, and admin experiences.
Data	PostgreSQL, Prisma or Drizzle, Redis	Relational marketplace data plus cache and rate limits.
Media	S3 compatible storage or Cloudinary	Product and shop images without storing files in the app server.
Later AI	Separate AI service interface, embeddings, vector store	Prevents AI work from complicating the marketplace MVP.

Delivery rules for every Codex task

- Work on one prompt at a time. Do not start the next phase until you have reviewed the previous phase.
- Before changing files, inspect the repository and preserve unrelated work.
- Use TypeScript, validation, pagination, ownership checks, and clear error responses.
- Run relevant lint, type-check, tests, and build at the end. Report exactly what passed and what remains.
- Never add checkout, online payment, shipping, or ERP/accounting features unless specifically requested.

- Use realistic but fictional Surat sample data. Never use real shop data without permission.

Phase 0 Product blueprint before code

Use this once, before any implementation. The deliverable is an architecture document, not code.

Prompt 0 Architecture and scope

You are the product architect for FindIt Surat, a Surat textile discovery platform. Do not write code. The product helps customers discover products and nearby Surat shops, contact sellers, receive a coupon, visit the shop, and buy offline. It is not e-commerce: no cart, payment, shipping, or checkout. Use React with TypeScript, React Router 7 framework mode SSR, Vite, Tailwind and shadcn/ui; Node Express TypeScript; PostgreSQL; Redis; object storage. Provide: 1) MVP scope centered on customer, seller, and admin; 2) recommended system architecture; 3) monorepo folders; 4) database entities and relationships; 5) API modules; 6) public, seller, and admin route map; 7) authentication and approval flow; 8) SEO plan; 9) modular future AI boundary; 10) phased delivery plan with acceptance criteria. Clearly label deferred features. Do not implement anything.

Phase 1 Foundation and design preview

Goal: create the project shell and a convincing clickable product preview using local mock data. This validates the product shape without committing to every backend feature.

Prompt 1 Create the React foundation

Implement only the FindIt Surat project foundation. Use a production-minded monorepo with apps/web (React TypeScript, React Router 7 framework mode SSR, Vite, Tailwind, shadcn/ui), apps/api (Express TypeScript), and shared packages for types/config. Configure PostgreSQL and Redis connection foundations, Docker Compose for local services, environment examples, ESLint, Prettier, TypeScript, logging, API v1 routing, error middleware, health endpoint, and README. Add public route shells for home, search, product, shop, category, and area; seller and admin route shells; and use local fixture data only for the preview. Make the home page mobile-first and visually feel like fashion discovery, not an ERP or e-commerce site. Do not implement business APIs, authentication, AI, checkout, payment, shipping, or external uploads. Run lint, type checks, and builds; fix failures. Stop after reporting results.

Prompt 2 Validate core user flows with mock data

Build a polished, responsive preview of the core FindIt Surat flows with local mock data only. Customer: browse home, categories, trending products, product detail, shop detail, search results and filters; every product card shows image, name, price/range, shop, area, discount, and View Shop. Product detail has Call, WhatsApp, Get Directions, and Get Coupon actions, but no Add to Cart. Seller: dashboard preview with shop profile, product list, add/edit product form, and simple inventory status. Admin: pending seller and product approval preview. Include 8 fictional Surat shops and at least 30 fictional textile products across Ring Road, Saroli, Vesu, Varachha, Udhna, and Katargam. Keep all actions visibly simulated and label them through UI state, not technical notes. Do not connect a database yet. Run checks and stop.

Phase 2 Marketplace MVP backend

Goal: replace mock behavior with the minimum real system needed to prove supply, discovery, and moderation.

Prompt 3 Database and seed data

Implement the PostgreSQL schema and migrations for the FindIt Surat marketplace MVP. Use UUIDs, timestamps, soft deletion where useful, explicit foreign keys, and indexes. Include User, SellerProfile, Shop, Product, ProductImage, Category, Subcategory, ProductTag, ProductAttribute, SellerVerification, ProductApproval, Favorite, CustomerEnquiry, Coupon, CouponRedemption, and lightweight Event records. Model seller-to-shop ownership, product-to-shop, product images, categories, approval status, product availability, and offline coupons. Index public product search/filter fields: category, subcategory, area, price, material, colour, pattern, discount, availability, seller/shop, approval, published state, created date, and popularity counter. Seed 8 fictional shops, 30 products, categories, and sample coupons. Run migrations and seed successfully, verify connectivity, and add concise setup notes. Do not build AI tables, quotations, CRM, advanced analytics, or payment.

Prompt 4 Authentication and approvals

Implement minimal secure authentication and authorization for CUSTOMER, SELLER, and ADMIN. Add registration, login, logout, current-user, password hashing, request validation, authentication middleware, and role/ownership authorization. Seller registration creates a pending seller profile; an admin must approve it before the seller can publicly operate. Add admin endpoints to approve or reject sellers and products. Protect seller and admin routes. Test customer registration, seller registration, admin login, anonymous rejection, and role restrictions. Do not add social login, MFA, password reset, or complex account settings yet. Run checks and stop.

Prompt 5 Seller shops and products

Implement the seller marketplace management MVP. An approved seller can create and edit only their own shops and products. Shop fields: name, description, business type, phone, WhatsApp, address, area, latitude, longitude, opening hours, and images. Product fields: name, description, category, subcategory, price/range, MOQ, material, colour, pattern, tags, availability, discount, multiple images, and publish status. Products must be approved by an admin before public visibility. Build validated REST APIs with pagination, filtering, sorting, ownership checks, and consistent errors. Connect the existing seller UI for real create, edit, publish/unpublish, and status views. Use a local development image strategy unless object-storage credentials are configured. Do not implement AI, inventory warehouses, CRM, quotations, or payments. Run tests and checks, then stop.

Prompt 6 Customer discovery and contact

Connect the public FindIt Surat React pages to real public APIs. Build indexed browse/search/filter pages and product/shop details for approved published products only. Filters: category, area, price, material, colour, pattern, discount, business type. Add favorites for signed-in customers. Implement contact actions: telephone link, WhatsApp deep link with a prefilled product enquiry, Google Maps directions deep link, and an enquiry form that creates CustomerEnquiry. Track lightweight events for product views, shop views, call clicks, WhatsApp clicks, directions clicks, and enquiry submission. Keep the experience mobile-first and discovery-led. There must be no cart, online checkout, payment, shipping, or add-to-cart UI. Test the complete seller-to-admin-to-customer path and stop.

Phase 3 Offline coupon and basic operations

Prompt 7 Offline coupon MVP

Implement the offline coupon MVP only. Admins and sellers can configure a percentage or fixed discount, expiry, and usage limit. A signed-in customer can generate a coupon linked to user, shop, optional product, discount, and expiry. Show code, discount, shop, expiry, and the message Show this coupon at the shop. Sellers can view generated, redeemed, and expired coupons for their own shops and manually mark a valid coupon as redeemed. Persist redemption time, seller, shop, and coupon. Enforce ownership,

expiry, and usage limits. Do not add POS, online payment, or external messaging integration. Add tests for generation, expiry, duplicate/limit handling, and redemption authorization.

Prompt 8 Basic seller and admin dashboards

Turn the existing seller and admin previews into the basic operational dashboards. Seller dashboard: product and shop views, enquiries, WhatsApp and call clicks, coupon redemptions, recent enquiries, top products, product management, simple availability status, and coupons. Admin dashboard: users, pending sellers, pending products, shops, product counts, enquiries, coupon usage, and moderation actions. Use the existing Event and marketplace tables; prefer simple aggregated queries and pagination. Do not build quotations, customer CRM, advanced charts, AI analytics, or a full ERP. Ensure responsive tables/cards and authorization. Run checks and stop.

Phase 4 Complete public SEO

SEO comes after real, approved public inventory exists. The public pages must be server-rendered; do not rely only on client-side meta tags.

Prompt 9 Public SEO implementation

Implement complete SEO for the public FindIt Surat React application using React Router 7 framework mode SSR. Index only approved, published, public products and shops. Create server-rendered titles, descriptions, canonical URLs, Open Graph/Twitter cards, robots.txt, sitemap index and scalable dynamic sitemaps for products, shops, categories, and areas. Add structured data where accurate: LocalBusiness for shops, Product with offline offer information where appropriate, BreadcrumbList, and WebSite/SearchAction. Build indexable routes such as /product/:slug, /shop/:slug, /category/:slug, and /area/:area/:category. Add sensible noindex rules for search result combinations, auth, seller, admin, duplicates, empty pages, and unapproved content. Optimize images with dimensions, alt text, lazy loading below the fold, and responsive sizes. Avoid keyword stuffing and do not claim shipping or online purchase. Validate generated metadata and structured data in tests or snapshots, run production build, and report the public URLs to inspect.

Phase 5 AI visual search after the marketplace works

AI is a separate service boundary. Start with an interface and mock provider so product work continues even without credentials. Only introduce a paid embedding provider and vector database after a measured pilot.

Prompt 10 AI service boundary and mock search

Add an independent AI search service boundary without tightly coupling it to Express. Define interfaces for image storage/processing, embedding provider, vector index, metadata filtering, and result ranking. Create AISearch and AISearchResult tables plus authenticated SearchHistory. Implement a mock provider that returns deterministic similar product results from the existing approved catalog; it must use the same API shape planned for real AI. Create POST /api/v1/ai-search accepting an image plus optional category, area, and price range, returning product, similarity score, shop, area, price, and image. Enforce file type, size, and image validation; rate limit the endpoint. Build no real OpenAI, LangChain, LangGraph, or vector database integration yet. Keep provider configuration in environment variables and document the replacement seam. Run tests and stop.

Prompt 11 AI search user experience

Build the premium mobile-first /ai-search experience against the mock AI API. Headline Find It From a Photo. Subtitle Upload a product, pattern or fabric image and discover similar products available across Surat. Support upload image and take photo where the browser supports it. Show preview, understandable

progress states (Analyzing image, Finding similar products, Matching Surat shops), errors, and Similar Products Found results. Each result shows image, product name, similarity percentage, shop, area, price, discount, and View Product, View Shop, Get Directions actions. Do not expose embeddings, vectors, RAG, LangChain, or LangGraph in customer copy. Preserve privacy with clear upload handling and deletion/retention configuration. Run checks and stop.

Prompt 12 Replace mock AI with a pilot integration

Before coding, review the existing AI interface and propose the smallest production pilot using the configured embedding provider and vector database. Explain cost, image retention, indexing workflow, backfill, retry behavior, monitoring, relevance evaluation, metadata filters, and rollback. Wait for approval. After approval, implement only the approved provider adapter and index pipeline, keeping the mock available for local development. Do not add a multi-agent AI system.

Phase 6 Deferred features after validation

Feature group	When to add it
Nearby discovery and maps	After shop locations are accurate and public browsing works.
Quotations and customer workflows	After sellers actively receive enquiries and ask for lightweight sales tools.
Advanced analytics and Redis counters	After baseline events show clear metrics worth monitoring.
WhatsApp Business API	After deep links produce enough qualified leads to justify integration.
Full AI semantic and image search	After the mock/pilot relevance evaluation meets an agreed quality threshold.
More inventory and ERP integrations	Only after clear repeated seller demand; keep FindIt focused on discovery and leads.

Final production hardening prompt

Prompt 13 Production review

Review the current FindIt Surat application as a senior production engineer. Do not add product features. Audit authentication, authorization, seller ownership, admin approval, input/file validation, rate limits, SQL injection, XSS, CSRF where applicable, secret handling, logging, error handling, database indexes, N+1 queries, caching, API response size, pagination, image optimization, mobile performance, accessibility, and public SEO. Run lint, type checks, tests, and production builds. Fix concrete failures. Update README, .env.example, Docker/deployment instructions, and security notes. Report results honestly; do not claim production readiness unless the checks pass.

Recommended decision gates

- Gate 1: Review the mock preview with 3 to 5 sellers and customers. Confirm that the discovery flow matches the intended business before database work.
- Gate 2: Confirm sellers can list products and admins can moderate them. This is the supply-side proof.
- Gate 3: Confirm customers can find, contact, and visit shops. Track contact actions and enquiries.
- Gate 4: Verify at least a useful base of approved product/shop pages exists before investing in SEO.
- Gate 5: Pilot AI search against a labeled set of product images and measure relevance before paying for scale.

Suggested definition of the first real milestone: 8 to 20 onboarded shops, a few hundred approved products, working phone/WhatsApp/directions actions, and a simple count of customer leads. That milestone tells you whether FindIt Surat is solving the discovery problem before you build the rest.